2-6. Kafka Architecture (Part1-5)

Thursday, 19 February 2026 12:12 PM

In EDA introduction part, we have already seen what is:

- Pub/Sub Model
- Streaming Model

And Advantages of EDA.

Today we will start with most popular Streaming Model : Kafka

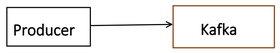
What is Kafka:

It's a distributed event streaming platform that allow us to:

- Publish events
- Store events
- Subscribe to events

Producer:

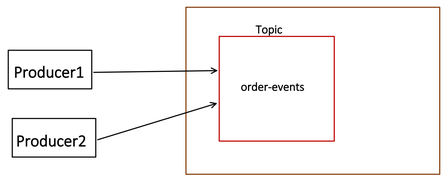
- Producer is an application that publishes events to Kafka.



- Producer does not care who will consume this event.

Topic:

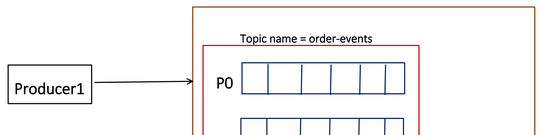
- Considered Topic as *category or Folder* to which events are published.

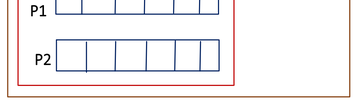


- Many Producers can write to same topic.

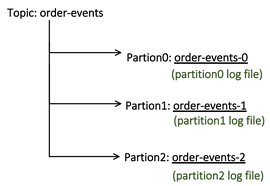
Partition:

- A Physical, ordered and append-only log that is part of a Topic.
- A topic can have many partitions.

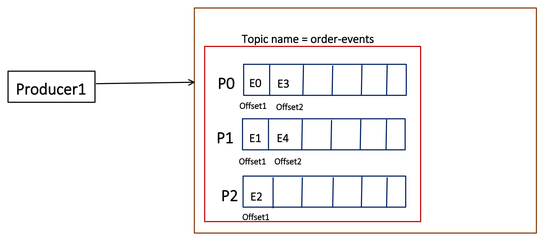




Physical: Events are actually stored on disk inside the partition.

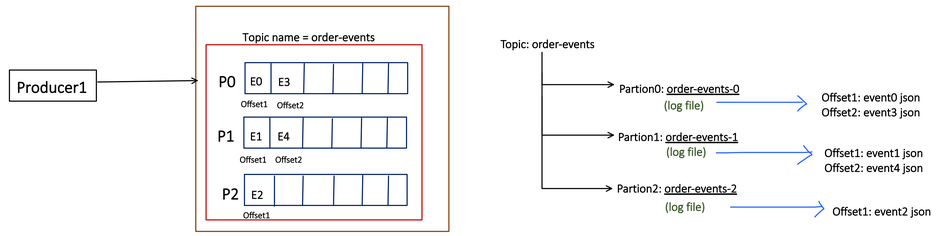


Ordered: Events are Written sequentially and assigned increment offsets. Consumer read them in offset order.

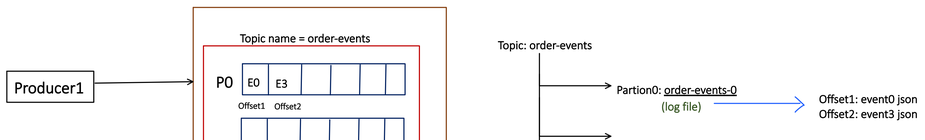


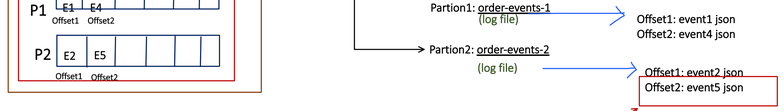
- Each partition maintains its *next offset counter.*
- When a new Event arrives:
  - Kafka assign next offset to that event.
  - And append the event to the log.
- Within a single partition - Order is guaranteed
  - Means events are written sequentially and read in increasing offset manner.
  - Kafka do not re-order the event in partition.
- But do not guarantees ORDER globally (across multiple partitions of the Topic)
  - For ex:
    - In Partiton0 (PO) : we know that Event3 happened after Event0 because offset2 comes after offset1 in that partition.
    - But I can not guaranteed that, Event4 in Partiton1 have happened after (or before) Event0 in Partition0. Because offset belongs to per

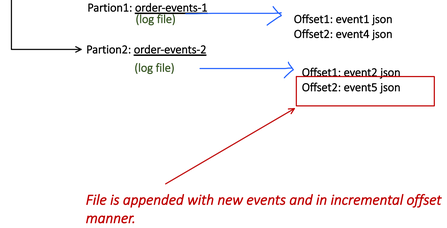
append-only log:



Let say, new event arrives E5 at Partiton2 (P2)



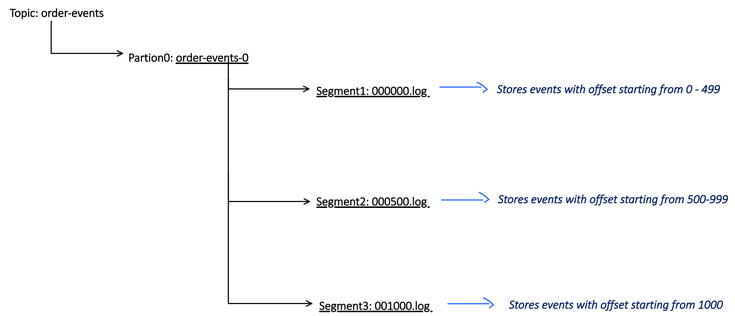




File is appended with new events and in incremental offset manner.

<u>If I go 1 level more deep:</u>

This partition log file is further divided into segments.

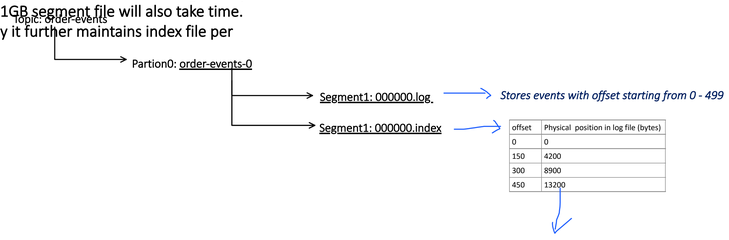


Why because:
Consumer can say, I want to read from Partition0 of order-events -> Offset 550 to 800.

If instead of 1 big log file, if logs are divided into multiple small segment files, searching can happen faster.

Kafka can directly jump to : <u>Segment2: 000500.log</u>

Now, we can control the segment size too, say if I keep the segment size as 1GB. Then again sequential search in 1GB segment file will also take time. That's why it further maintains index file per segment.



offset	Physical position in log file (bytes)
0	0
150	4200
300	8900
450	13200

Not every offset is indexed.
Instead it index every N bytes, means after N bytes have been written to log file, it creates 1 index row.

This saves memory and also helps in faster lookup.

Example:

<u>Partition0 of Order-events:</u>
Offset0- Event size (300 bytes)
Offset1- Event size (500 bytes)
Offset2- Event size (1000 bytes)
Offset3- Event size (2000 bytes)
Offset4- Event size (500 bytes)

<u>Segment size</u>: 1GB
<u>Index Interval bytes</u>: 4096 bytes (means after this much bytes written to log file, we need to create one index record in .index file)

Offset	Evet size	File position
0	300	0
1	500	300
2	1000	800
3	2000	1800
4	500	3800

Now total bytes written : 300+500+1000+2000+500 = 4300 bytes

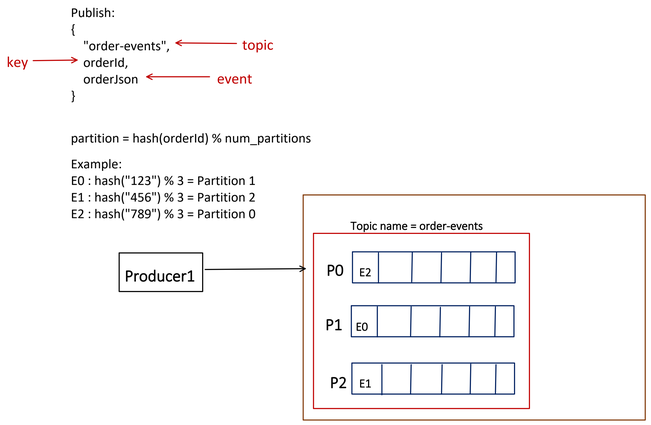
4300 Segment file 00000.index (4096), so we need to create 1 record in index file.

offset	Physical position in log file (bytes)
4	3800

Producer can pick one of the different algorithms to decide which partition of a topic an event should be sent to.

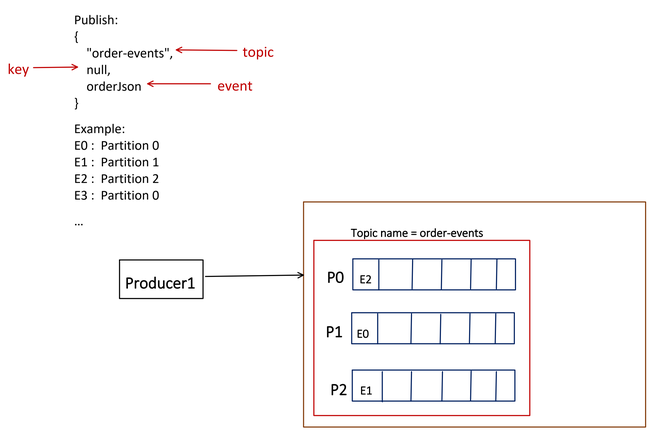
1. Key-based Partitioning

All events for same KEY will goes to same partition (ordering guaranteed)



2. Round Robin (No key)

Even distribution but ordering is not guaranteed.
Related events may go to different partitions.



Custom Partitioning

We can also write our own custom logic say:

country is "India"-> got to Partition0
country is "US"-> got to Partition1

During implementation, we will see how to write the custom partitioning logic.

Broker:

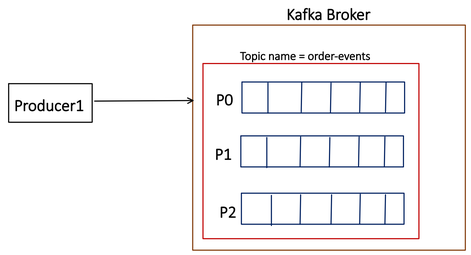
A broker is single Kafka server instance.
A Broker is the one which actually stores data and servers client (Producer and Consumer).
A Broker stores some partitions of some topics.

Important Note:

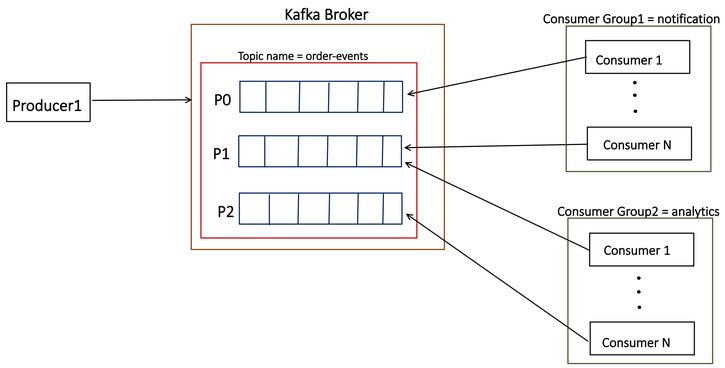
Topics and Partitions are distributed across multiple brokers. That's why I said 1 broker stores only some partitions of some topics.

- *So, It do not hold all topics.*
- *also, It do not holds all partitions of the topic*

We will understand multiple broker scenario, when we will see Kafka cluster below (later part of the topic)



Consumer and Consumer Groups:



Rule:

In one consumer group: Same partition is not read by multiple consumers.

Scenario 1: Consumers = Partitions

Topic: order-events (3 partitions)
Consumer Group: notification-service (3 consumers)

Assignment:
Consumer 1 → Partition 0
Consumer 2 → Partition 1
Consumer 3 → Partition 2

Scenario 2: Consumers < Partitions

Topic: order-events (6 partitions)
Consumer Group 1: notification-service (3 consumers)

Assignment:
Consumer 1 → Partition 0, Partition 3
Consumer 2 → Partition 1, Partition 4
Consumer 3 → Partition 2, Partition 5

Scenario 3: Consumers > Partitions

Topic: order-events (3 partitions)
Consumer Group 1: notification-service (5 consumers)

Assignment:
Consumer 1 → Partition 0
Consumer 2 → Partition 1
Consumer 3 → Partition 2
Consumer 4 → IDLE (no partition)
Consumer 5 → IDLE (no partition)

Multiple consumer groups: Same partition can be read by multiple consumers present in different group.

Topic: order-events

Consumer Group 1: notification-service
Consumer 1 → Partition0

Consumer Group 2: analytics-service
Consumer 2 → Partition 0

Consumer Group 3: audit-service
Consumer 3 → Partition 0

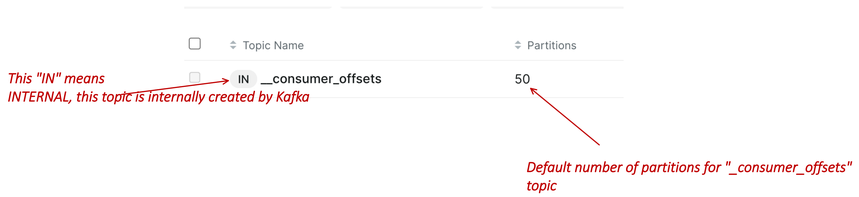
Each Consumer group maintains partition wise offset independently.

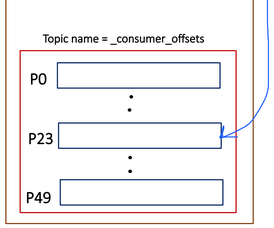
(Consumer group, topic, partition)- Offset consumed till now

For example:

Consumer Group	Topic	Partition	Committed Offset (processed up to this offset)
notification	order-events	**0**	**105**
notification	order-events	1	**98**
analytics	order-events	**0**	**60**
analytics	order-events	**1**	**45**

These Consumer groups offset information is stored in Kafka *"_consumer_offsets"* topic





{
"_consumer_offsets",
notification_group_id,
message:(notification_group_id, topic, partition, committed_offset)
}

Compute the partition = hash(notification_group_id) % 50 = 23

All offset update for consumer group: Notification, will goes to Partition23 of topic: **_consumer_offsets**

This committed offset strategy, helps when consumer got crashed and say new consumer comes up.
it finds out the partition no of topic "_consumer_offsets"

hash(notification_group_id) % 50 = 23

Now, Consumer will ask the Broker which hold this Partition23 of topic "_consumer_offsets".

Till what point events are processed for say: TopicA-Partition2

Broker, internally maintains the key-value map from the "_consumer_offsets" metadata file
Key: topic, partition
value: committed offset

Broker checks the map and return the committed offset value.

Offset Commit Strategies:

Strategy 1: Auto-Commit

auto.commit.interval.ms=5000

Means, auto commit every 5 seconds.

But this strategy is Risky, why lets see an example:

Time 0s: Consumer polls, gets events 0-99
Time 1s: Consumer processing events...
Time 5s: Auto-commit triggers -> commits offset 99
Time 6s: Consumer crashes (only processed 50 events)
Time 7s: Consumer restarts from offset 100

Result: Events 51-99 are LOST (at-most-once delivery)

Strategy 2: Manual Commit

Step 1: Poll events 0-99
Step 2: Process ALL events
Step 3: Commit offset 99 (wait for confirmation)
Step 4: Kafka confirms commit
Step 5: Continue to next batch

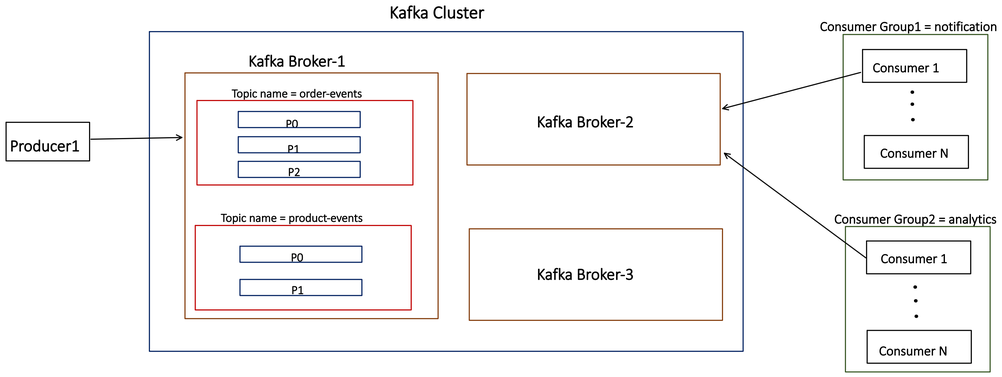
If crash happens:
- Before commit: Events reprocessed (at-least-once)
- After commit: Events not reprocessed

lets move to Advanced level of info:

Kafka Cluster:

Kafka cluster is a group of Brokers working together to provide:

Scalability : distribute load across multiple servers
Fault tolerance: continue operation even if Brokers fails
High availability: No single point of failure.



Now question is How? How this Multi-Broker architecture helps to achieve: Scalability, Availability and High tolerance

Leader-Follower Partition

For each partition-> One broker is Leader and others are followers

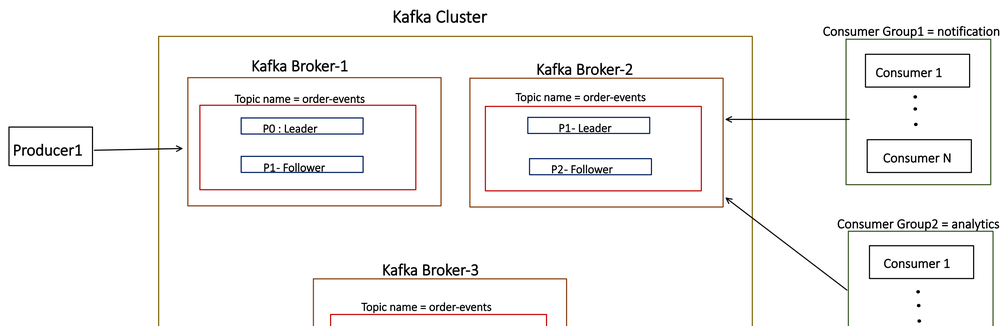
Example:

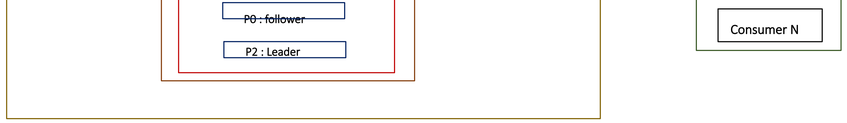
Topic: order-events
Partitions: 3
Replication Factor: 2

3 Partitions of topic "order-events":
- P0
- P1
- P2

Each partition has 2 replication:
- P0 (one leader and one follower)
- P1 (one leader and one follower)
- P2 (one leader and one follower)

All these are distributed among the brokers





Leader Responsibility:

Handle all Producer Writes
Handle all Consumer Reads
Maintains Partition logs
Coordinate with followers

Follower Responsibility:

Replicate data from Leader
Stay in sync with Leader
Ready to become Leader if current leader fails
Do not serve client (Producer or Consumer) requests

Now, 1 question might have comes to your mind

Who decides:
Which Broker hosts which Partition replica?
Which partition replica become leader?
Which partition replica stays as follower?

Answer is: Controller

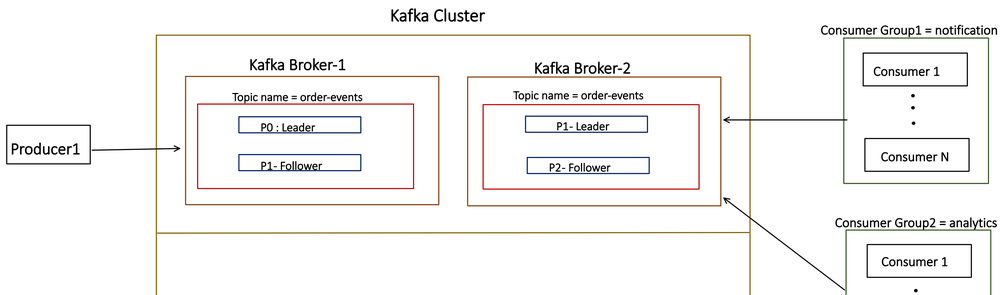
It's a special broker which responsibilities are:

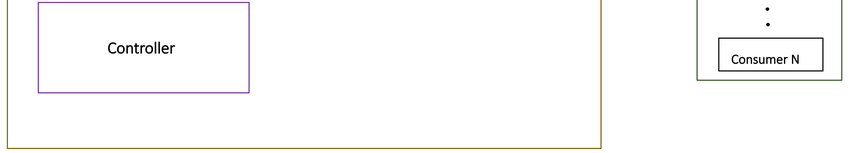
Creation of Topic
Elect partition leaders and followers
Detect Broker failures (using heartbeats)
Notify all Brokers about the changes

In short, Controller manages the Cluster metadata (info about topics, partitions, broker etc..)

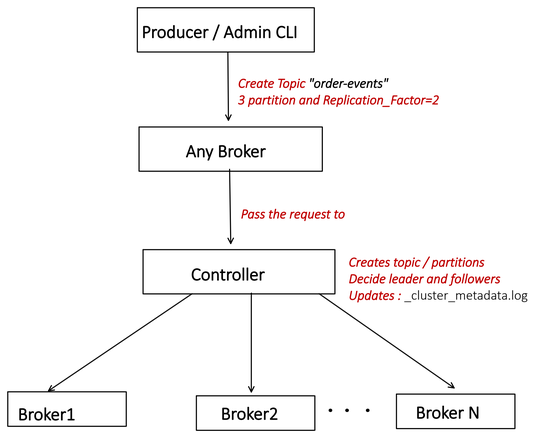
A Controller node can either :
have dual responsibility : Normal Broker + Controller Role
or
have just 1 responsibility : Controller Role

For simplicity:
I am taking example with Controller node which just have 1 responsibility i.e Controller Role





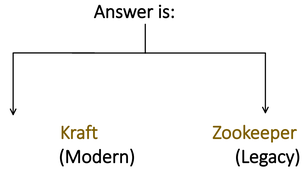




Now again, knowing about Controller, few things comes to our mind:

What if this controller get failed? Seems like a brain of the cluster itself.

Okay, even if answer is: we can have multiple Controller, but who will control them. Like we have multiple Brokers and we know Controller helps them in their coordination.
But what if there are multiple Controller, who will bring coordination among them? Do we need another controller for the controllers? So it will become a loop?



I will not cover Zookeeper, as it becomes deprecated in latest versions of Kafka (>=3.x). And its been replaced with KRaft. Both helps Kafka to manage cluster metadata (topics, partition, broker, controller election).

- Zookeeper is an external distributed system, which Kafka has to:

Deployed separately
Monitored separately
And maintained separately from Kafka

Whereas KRaft (Kafka Raft) is inbuilt in Kafka. So less overhead to manage an external system.

Both Zookeeper and KRaft uses Consensus algorithm:

Zookeeper: uses Zab Consensus
KRaft: uses Raft Consensus

Problem:

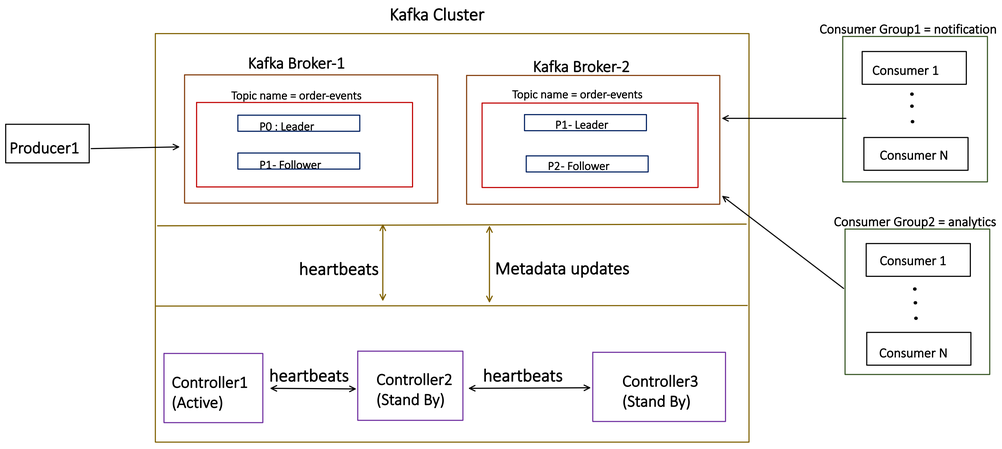
1. Controller is bottleneck for Kafka cluster.

Lets understand first, what is Consensus and why Controller in Kafka needs it?

Why?
Because Controller are the ones:
that stores the Cluster metadata.
Creates/Delete topic
Decide Partition Leader and Followers
Keep track of Broker heartbeats and timely detect any Broker failure.
If any Broker is failed and distributed its Topic and Partition to other Brokers.
Keep other Brokers up to date about the cluster information.

So, if this Controller fails, then Cluster is unavailable. We can not create new Topics, if any broker is down, no one will be able to take its partition responsibility.
All cluster metadata information is lost.

So, we need Multiple Controllers



And only 1 controller is Active (Leader) in 1 Cluster. Others controllers are act as Followers (Stand By).

Now question is: Who will decide among Controllers, who is Leader and who is Follower?

We can not have another Controller to control these Controllers (so it will become a

loop). That's where KRaft (*implements RAFT Consensus algorithm*) comes into picture.

In this Consensus algorithm: All decision is taken by QUORUM.

Quorum means: Majority of the nodes.

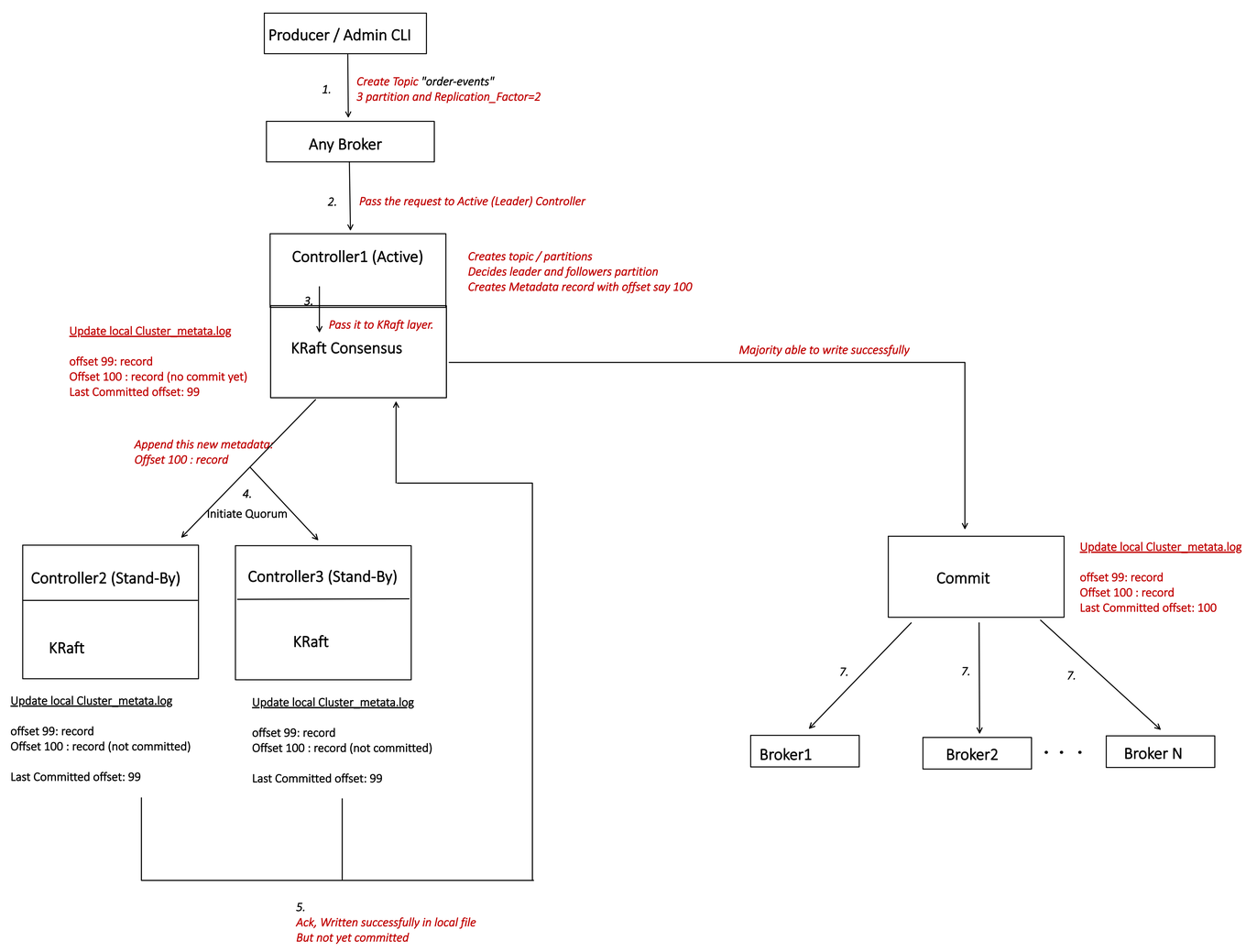
lets say we have 3 controllers, so Quorum would be: (N/2)+1

(3/2)+1 = 2

So these many controller nodes should agree on any decision before it can be final.

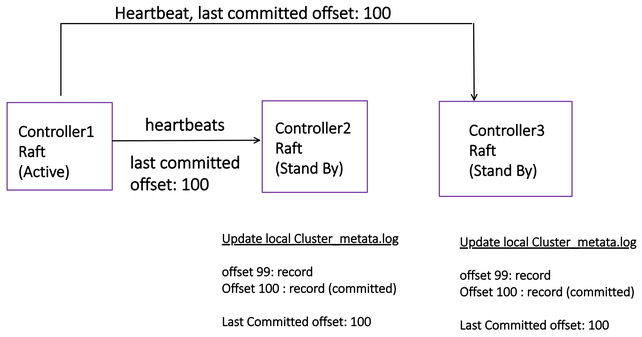
For example, Quorum happens on:

Which controller node should becomes the leader (Active) controller.
Any change in Cluster Metadata information.
Etc.



And when Active Controller send its heartbeat to Stand-by Controllers, it also

send the last committed offset. So that they can also update their local cluster metadata logs.



ISR (In-Sync Replica)

Controller also maintains one more important Cluster Metadata i.e ISR

ISR: For each Partition, ISR represents the list of replicas which are fully caught up with leader.

Example:

Topic: order-events
Partition: 1
Replication factor = 3

Leader: Broker 1
Followers: Broker 2, Broker 3

Possible ISR:

ISR = {Broker 1, Broker 2, Broker 3}

If say Broker 3 lags too much:

Broker1(Leader):[0][1][2][3][4][5][6][7][8]
[9] ← Latestoffset:9
Broker2(Follower):[0][1][2][3][4][5][6][7][8]
[9] ← In-Sync
Broker3(Follower):[0][1][2][3][4][5][6] ←
Out-of-Sync

ISR={Broker1,Broker2}

Broker 3 is temporarily removed from ISR.

Now the question is, how Broker (Leader) decides particular follower is out of sync.

A follower is considered in-sync if:
- Its replica offset is close enough to leader's log end offset.
- It stays within replica.lag.time.max.ms

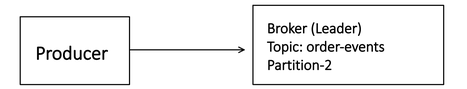
If a follower:
- Stops fetching
- Or is too slow

It gets removed from ISR.

Once leader identifies that particular follower broker is out of sync, it request

Controller to alter the ISR

What is the use of ISR?



Publish:
{
"order-events",
null,
orderJson
}

Also set the config for events to be publish:

ack = 0 or 1 or all

Persist the event in
Partion-2 logs (one of its segment file)

Now Producer has used : ack=all

So, Broker (Leader) will wait for all ISR replicas to update the events. Then only it sends successfully publish event to Producer.

Possible ACK values:

0 - Fire and Forget
Producer sends record to leader.
Producer does **not wait** for any response.
Producer immediately considers the send successful.

1 - Wait for Leader
Producer sends record.
Leader writes record to its local log.
Leader sends ACK back.
Producer waits for that ACK.

All - Wait for all ISR replicas to write

Now, question might comes to you mind, what if ISR list becomes empty?

It can not happen: there is also 1 more cluster metadata information

min.insync.replicas=2

Which tells, minimum number of replicas should be present in ISR.

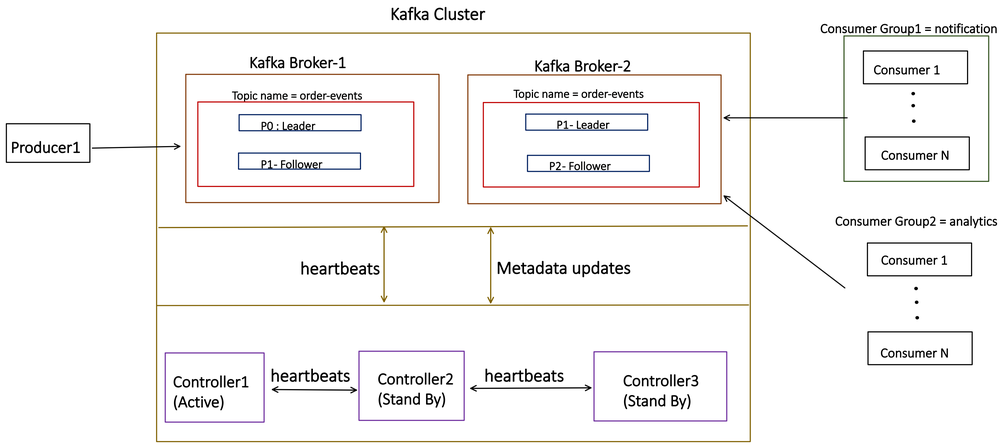
If counts goes less then this, Partition is UNAVAILABLE.

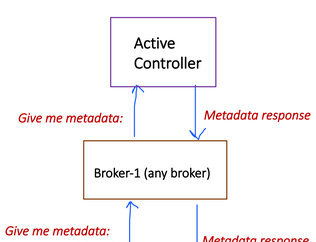
Now lets cover:
Complete flow, combining all the information till now

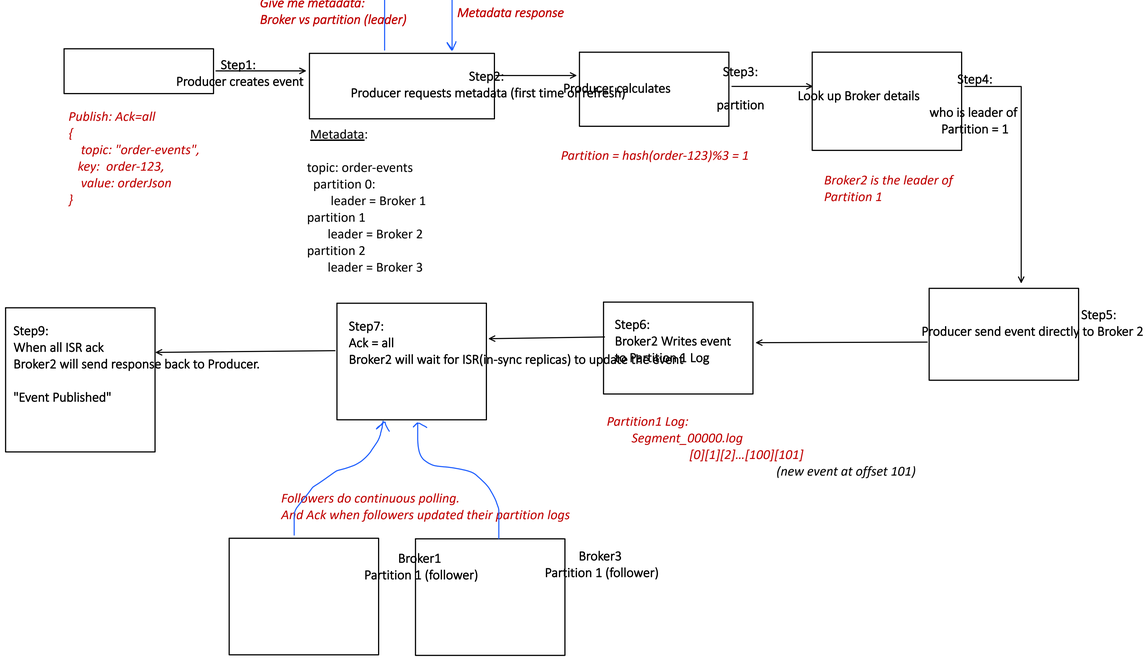
Now, Topics are created, Leader-Follower partition is decided, Brokers are updated

with Cluster Metadata.
Now lets see:

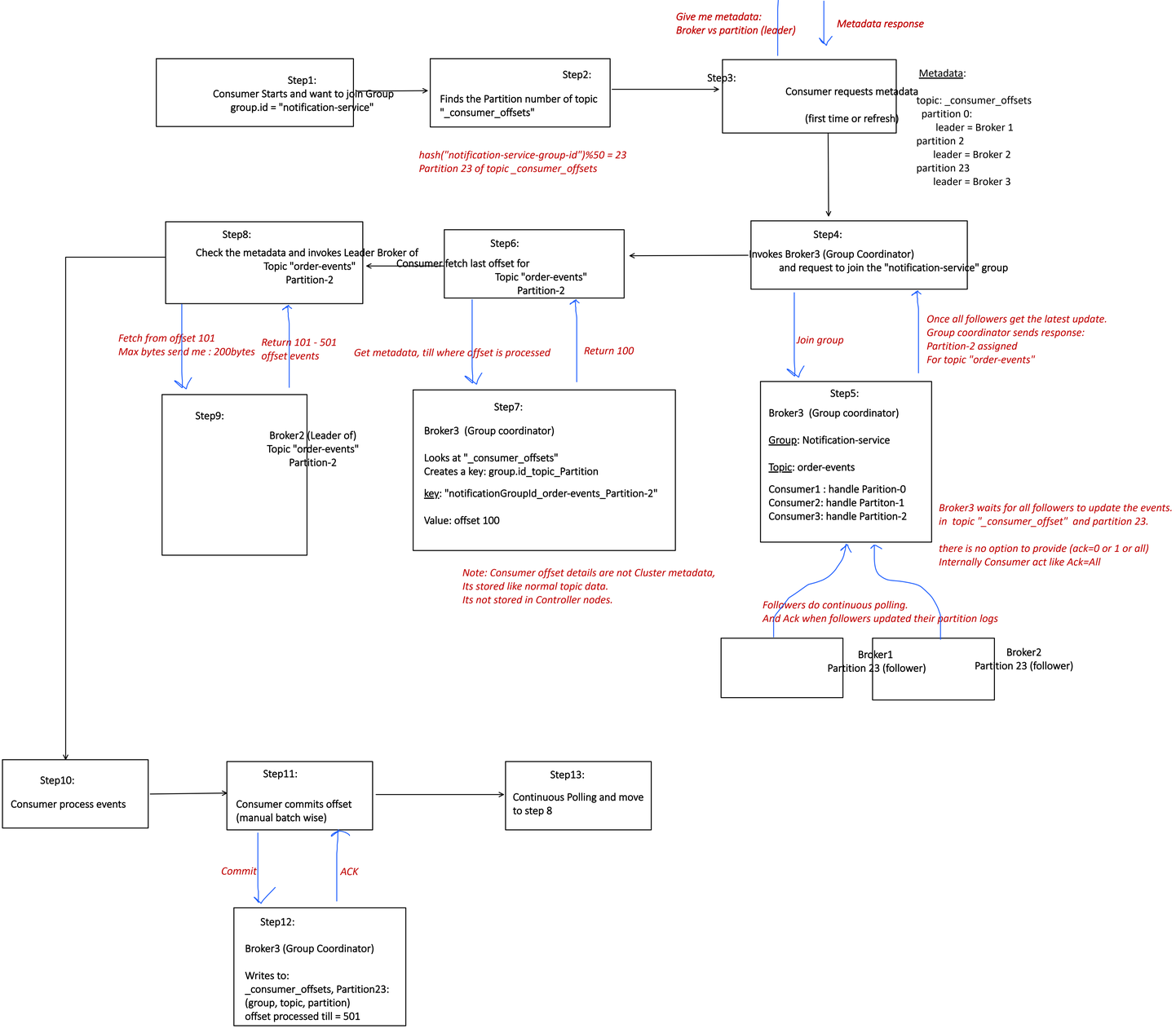
Complete Producer Write Flow:





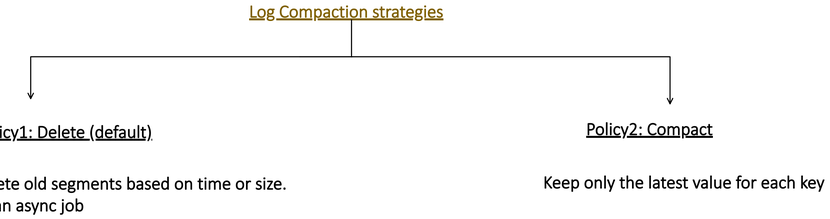


Complete Consumer Read Flow:



Note:

From above flow, one thing we get to know is, Consumer follows PULL model. Kafka do not follow PUSH model.



Configuration:
cleanup.policy=delete
retention.ms=604800000 ← 7 days
retention.bytes=1073741824 ← 1GB

Behavior:

Time based:
Segment1:Created 8 days ago -> DELETED
Segment2: Created 6 days ago -> KEPT

Size based:
Total partition log size becomes more than 1GB
Segment1: 1GB
Segment2: 200MB

Total 1.2GB
So kafka will delete old segment Segment1

Configuration:
cleanup.policy=compact

Segment:

offset	key	record
100	user1	v1
101	user2	v1
102	user1	v2
103	user3	v1
104	user2	v2

After cleanup:

offset	key	record
102	user1	v2
104	user2	v2
103	user3	v1

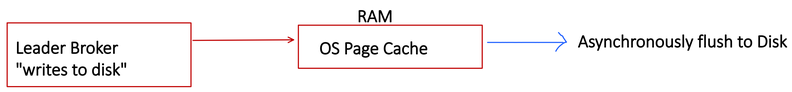
Compaction:
- Happens at segment level
- Is asynchronous
- Does not block writes
- Does not guarantee immediate cleanup

1 doubt might comes to your mind:

Kafka is reading from and writing to DISK, isn't disk reads and writes are slow. Then how come Kafka is so fast

Page Cache Optimization for Event writes

Kafka trusts OS for caching the log files efficiently.



No blocking disk wait
Sequential write (best case for disks)
- Because kafka is append only:
  - No insert in mid
  - Never modifies old data
  - Never update record in place
  - It only writes at end.
- So disk keeps moving forward.

Zero-Copy Optimization for Event Reads

When we read something from Disk, flow is:

Disk-> OS Page Cache-> User Space-> Network

In case of Kafka, it says, do not bring data to my space, send directly to network. Uses sendfile() system call.

Disk-> OS Page Cache-> Network

No copy is maintained in Kafka JVM.

Advantages:

Data copy time and memory saved
CPU cycles saved
High throughput

Edge Cases and Failure Scenarios:

What happens when Active Controller fails:

Initial State:
Controller1: Leader
Controller2: Follower
Controller3: Follower

Controller1 crashes:
Controller2 & 3: No heart beat from leader

Election starts:
Controller2:Starts election, requests votes
Controller3: Votes for Controller2

Controller2 wins:
Controller2: Becomes new leader(2/3majority)
Controller3: Remains follower

Impact: New leader elected, cluster continues

What happens when Leader Partition fails:

Setup:

Partition 0 (RF=3)
Leader: Broker1
Follower: Broker2
Follower: Broker3

What Happens:

Step 1: Broker 1 (leader) crashes

Step 2: Controller detects leader failure, as there is no heartbeat from leader

Step 3: Controller triggers leader election
Candidates: Broker 2, Broker 3 (both in ISR)
New Leader: Broker 2 (selected from ISR)

Step 4: Controller updates metadata
Leader: Broker 1 -> Broker 2
ISR: {Broker 2, Broker 3}

Step 5: Clients receive metadata update
Producers/Consumers reconnect to Broker 2

Step 6: System resumes normal operation

Impact:
NO data loss (new leader has all committed data)
Automatic recovery

What happens when Follower Partition fails:

Setup:

Partition 0 (RF=3)
Leader: Broker 1
Follower: Broker 2
Follower: Broker 3

What Happens:

Step 1: Broker 3 crashes

Step 2: Leader (Broker 1) detects Broker 3 is unresponsive, no timely fetch calls

Step 3: Controller removes Broker 3 from ISR
ISR: {Broker 1, Broker 2, Broker 3} -> {Broker 1, Broker 2}

Step 4: System continues normally
- Producers still write (acks=all waits for 2 replicas)
- Consumers still read
- NO downtime

Step 5: Broker 3 recovers

Step 6: Broker 3 catches up with leader

Step 7: Broker 3 added back to ISR
ISR: {Broker 1, Broker 2} -> {Broker 1, Broker 2, Broker 3}

Impact:

NO data loss
NO downtime

What happens when Topic fails:

When interviewer ask this question, my answer would be:

Topic never fails, as it is just a logical category or folder. Partition fails, which we have seen above.

What happens when Consumer fails:

Scenario1: Consumer Crashes Before Committing Offset

What Happens: Manual Commit (batch wise)

Step 1: Consumer polls events (offsets 100-199)
Step 2: Consumer processes events 100-150
Step 3: Consumer CRASHES (before commit)
Step 4: Consumer restarts
Step 5: Consumer asks Kafka: What's my last offset?
Step 6: Kafka responds: Offset 99 (last committed)
Step 7: Consumer reprocesses events 100-150 (DUPLICATES)

Result: At-least-once delivery (duplicates possible)

Scenario2: Consumer Heartbeat timeout

What Happens during Long Processing Time or when Consumer fails:

Step 1: Consumer polls events
Step 2: Consumer starts processing (takes 6 minutes)
Step 3: max.poll.interval.ms (5 minutes) exceeded
Step 4: Group Coordinator considers consumer DEAD, as it did get any heartbeat (consumer is busy in processing)
Step 5: Rebalance triggered
Step 6: Partitions reassigned to other consumers
Step 7: Original consumer finishes processing
Step 8: Original consumer tries to commit
Step 9: Group coordinator rejects commit (consumer not in group)

Result: Wasted processing

Solution:
1. Increase max.poll.interval.ms
2. Reduce max.poll.records (process smaller batches)
3. Optimize processing logic (faster processing)
4. Use separate thread pool for processing

What happens when Broker fails:

Setup:

Cluster: 3 brokers
Topic: order-events (6 partitions, RF=3)

What Happens:

Step 1: Broker 1 crashes

Step 2: Controller detects failure, as it did not receive any heartbeat.

Step 3: For each partition where Broker 1 was leader:
- New leader elected from ISR
- Metadata updated

Step 4: For each partition where Broker 1 was follower:
- Removed from ISR
- Partition continues with remaining replicas

Step 5: Clients receive metadata update

Step 6: System continues normally

Impact:
- NO data loss
  Automatic recovery